Go · Cheatsheet de bases

Resumen del proyecto Bases-go: tipos, flujo, funciones, punteros, slices, mapas, interfaces, errores, JSON y concurrencia

1 Comandos de Go

```
go mod init nombre    // crea go.mod
go mod tidy           // sincroniza dependencias
go get pkg@version    // agrega una dependencia
go run .              // compila y ejecuta
go build              // genera el binario
go test ./...         // corre los tests
go vet ./...          // busca errores sospechosos
gofmt -w .            // formatea el código
go run -race .        // detecta data races
```

- Una carpeta = un paquete. `main` + `func main()` es la entrada.
- Mayúscula inicial = exportado. Minúscula = privado al paquete.
- Paquetes en minúscula y una palabra: `tiposdedatos`.

2 Variables y tipos

```
var x int = 10        // forma larga
y := 10               // corta (solo dentro de funciones)
const Pi = 3.14
var a, b = 1, "hola"

float64(i)            // conversión explícita
int(3.14)             // trunca: da 3
strconv.Itoa(5)       // int -> "5"
n, err := strconv.Atoi("5")   // "5" -> int
```

`bool`	`true` / `false`
`string`	texto inmutable (UTF-8)
`int int8..int64`	enteros (`int` depende de la arquitectura)
`uint uint8..uint64`	enteros sin signo
`float32 float64`	decimales (`float64` por defecto)
`byte / rune`	alias de `uint8` / `int32` (un carácter Unicode)
`complex64/128`	números complejos

- Valor cero: `0`, `0.0`, `""`, `false`, `nil`.
- Los bits (8,16,32,64) son tamaño/precisión, no arquitectura.

3 fmt: imprimir y formatear

```
fmt.Println("a", x)          // espacios + salto de línea
fmt.Printf("%s: %d\n", n, e)  // NO agrega \n solo
s := fmt.Sprintf("%03d", 7)   // "007" (devuelve string)
err := fmt.Errorf("falla: %w", e)
fmt.Fprintln(os.Stderr, "error")
fmt.Scan(&x)                  // lee entrada (con &)
```

`%v %+v %#v`	valor / con campos / sintaxis Go
`%T`	tipo del valor
`%d %s %q`	entero / string / string con comillas
`%f %.2f`	decimal / 2 decimales
`%t %x %b %c`	bool / hex / binario / carácter
`%p %%`	puntero / signo %
`%5d %-5d %05d`	ancho / izquierda / con ceros

- `Println` ya separa con espacio: no pongas `": "`.
- Si un tipo tiene `String() string`, fmt lo usa solo.

4 Control de flujo

```
if n < 18 {
} else if n < 65 {
} else {
}
if v, ok := m["a"]; ok { }   // init; condición

for i := 0; i < 5; i++ { }   // clásico
for n < 3 { }                // tipo while
for { break; continue }      // infinito
for i, v := range s { }      // índice y valor
for _, v := range s { }      // ignora el índice
for i := range 3 { }         // 0 1 2 (Go 1.22+)

switch x {
case 1:                      // sin break
case 2, 3:                   // varios valores
default:
}

defer fmt.Println("al salir")  // se ejecuta al terminar
```

- Varios `defer` corren en orden inverso (LIFO).
- `range` recorre slices, arrays, mapas, strings y canales.

5 Funciones

```
func suma(a, b int) int { return a + b }

func dividir(a, b int) (cociente, resto int) {
   cociente = a / b
   resto = a % b
   return                    // retorno con nombre
}

func buscar(b int) (int, error) {   // error al final
   if b == 0 {
      return 0, errors.New("div por 0")
   }
   return 10 / b, nil
}

func nombres(n ...string) { }     // variádica
nombres("a", "b")

func aplicar(n int, f func(int) int) int {
   return f(n)                // función como valor
}
aplicar(3, func(x int) int { return x * 2 })

func contador() func() int {      // closure
   c := 0
   return func() int { c++; return c }
}
cont := contador()   // cont() -> 1, 2, 3
_, err := buscar(0)  // _ ignora un valor
```

- Go siempre pasa copias de los argumentos.

6 Punteros

```
p := &x            // & = dirección de x
*p = 5             // *p = valor al que apunta
q := new(int)      // puntero a int con valor 0

func inc(v *int) { *v++ }
inc(&x)            // ahora sí modifica x
```

- `*int` en un tipo = "puntero a int". `*p` en una expresión = desreferenciar.
- Desreferenciar un puntero `nil` causa panic.
- Slices, mapas y canales ya comparten sus datos al pasarlos.

7 Structs y métodos

```
type Rect struct {
   Ancho, Alto float64
}
r := Rect{Ancho: 10, Alto: 20}
r.Alto = 5

func (r Rect) Area() float64 {      // copia: no modifica
   return r.Ancho * r.Alto
}
func (r *Rect) Escalar(f float64) { // puntero: SÍ modifica
   r.Ancho *= f
   r.Alto *= f
}
r.Escalar(2)         // Go toma &r solo
```

- No hay clases: un método es una función con receptor.
- Si algún método muta, usa receptor puntero en todos.

8 Arrays y slices

```
a := [3]int{1, 2, 3}        // array: tamaño FIJO
s := []int{1, 2, 3}         // slice: crece con append
s = append(s, 4, 5)         // SIEMPRE reasigna
len(s)  cap(s)              // elementos / capacidad

sub := s[1:3]               // [inicio:fin) COMPARTE memoria
sub := s[1:3:3]             // limita la cap: append no pisa
make([]int, 3)              // len 3 ya con ceros
make([]int, 0, 10)          // len 0, cap 10
copy(dst, src)              // copia real
copia := slices.Clone(s)

s = append(s[:i], s[i+1:]...)   // borrar índice i
s = slices.Delete(s, i, i+1)
s = slices.Insert(s, i, 99)
slices.Contains(s, 8)  slices.Index(s, 8)
slices.Sort(s)  slices.Reverse(s)
slices.Equal(a, b)          // slices no usan ==
slices.Min(s)  slices.Max(s)

var n []int      // nil: JSON -> null
v := []int{}     // vacío: JSON -> []
m := make([][]int, 3)       // matriz: crea cada fila
```

- `[3]int` es array, `[]int` es slice (no es un "array dinámico").
- `append` puede reservar memoria nueva: por eso se reasigna.

9 Mapas

```
m := map[string]int{"a": 1}
m := make(map[string]int)   // vacío y listo
m["b"] = 2                  // agregar / actualizar
v := m["z"]                 // no existe -> valor cero
v, ok := m["z"]             // ok=false si no existe
delete(m, "b")              // borrar (no falla si no está)
len(m)
for k, v := range m { }     // orden ALEATORIO
ks := slices.Sorted(maps.Keys(m))   // claves en orden
copia := maps.Clone(m)
clear(m)
set := map[string]struct{}{}        // conjunto
g[k] = append(g[k], x)              // agrupar
cuenta[letra]++                     // contar
```

- Mapa `nil`: leer funciona, escribir hace panic.
- Son referencia: `b := a` comparte el mismo mapa.
- `m["a"].Campo = x` no compila: reasigna el struct o usa `*T`.
- Claves comparables (nunca slices, mapas o funciones).
- No son seguros entre goroutines: protege con mutex.

10 Interfaces

```
type Forma interface {
   Area() float64
}
// Se cumple solo con tener el método: no hay 'implements'
formas := []Forma{Circulo{5}, Cuadrado{4}}

var _ Forma = Circulo{}      // verifica al compilar

switch v := f.(type) {       // type switch
case Circulo:
   fmt.Println(v.Radio)
default:
   fmt.Printf("%T\n", v)
}
c, ok := f.(Circulo)         // assertion segura

type FormaCompleta interface { // composición
   Forma
   Perimetro
}
var x any = 42               // interfaz vacía
```

- Si `Area` tiene receptor `*T`, solo `&T{}` cumple la interfaz.
- `error` y `fmt.Stringer` son interfaces de 1 método.
- Interfaces chicas y definidas donde se usan.

11 Errores

```
var ErrNoEncontrado = errors.New("no encontrado")

type ErrValidacion struct{ Campo string }

func (e *ErrValidacion) Error() string {
   return "falta " + e.Campo
}

return fmt.Errorf("buscar %d: %w", id, err)  // envolver

errors.Is(err, ErrNoEncontrado)   // centinela (no uses ==)
var ev *ErrValidacion
errors.As(err, &ev)               // saca el tipo propio
errors.Unwrap(err)                // abre una capa
errors.Join(e1, e2)               // junta varios

resultado, err := hacer()
if err != nil {                   // salir temprano
   return err
}
```

- Mensajes en minúscula y sin punto final.
- Agrega contexto con `%w`; no loguees y devuelvas a la vez.
- Trampa: devolver un `*T` nil como `error` NO es nil.
- Patrón API: `errors.Is/As` -> código HTTP (400/404/500).

12 JSON

```
type User struct {
   ID    int    `json:"id"`
   Email string `json:"email,omitempty"` // omite si ""
   Edad  *int   `json:"edad"`    // nil -> null
   Clave string `json:"-"`       // nunca se convierte
   priv  string                  // minúscula: ignorado
}

b, err := json.Marshal(u)            // struct -> []byte
b, _ = json.MarshalIndent(u, "", "  ")
err = json.Unmarshal(b, &u)          // OJO: con &

json.NewEncoder(w).Encode(u)         // a un io.Writer
json.NewDecoder(r.Body).Decode(&u)   // desde un io.Reader
dec.DisallowUnknownFields()          // modo estricto

var m map[string]any                 // JSON sin estructura
m["edad"].(float64)                  // números: float64
```

- Solo se convierten campos exportados (Mayúscula).
- Campos desconocidos se ignoran, salvo modo estricto.
- `[]T(nil)` da `null`; `[]T{}` da `[]`.

13 Concurrencia

```
go funcion()                  // lanza una goroutine
ch := make(chan int)          // sin búfer: sincroniza
ch := make(chan int, 3)       // con búfer de 3
ch <- 1                       // enviar
v := <-ch                     // recibir
v, ok := <-ch                 // ok=false: cerrado y vacío
close(ch)                     // cierra quien ENVÍA
for v := range ch { }         // hasta que cierre

func f(out chan<- int, in <-chan int) { } // direcciones

var wg sync.WaitGroup
wg.Add(1)
go func() {
   defer wg.Done()
}()
wg.Wait()                     // espera sin Sleep

select {
case m := <-ch:
case <-time.After(time.Second):   // timeout
}
```

- Una goroutine NO es un hilo del SO: es ligera, puede haber miles.
- Deadlock: `all goroutines are asleep` (nadie recibe/envía).
- Enviar a un canal cerrado hace panic.

14 Mutex y atomic

```
var mu sync.Mutex
mu.Lock()
defer mu.Unlock()             // libera aunque haya return
n++

var rw sync.RWMutex           // muchos lectores o 1 escritor
rw.RLock();  defer rw.RUnlock()   // leer
rw.Lock();   defer rw.Unlock()    // escribir

var c atomic.Int64            // contador simple
c.Add(1);  c.Load()

type Contador struct {        // mutex junto al dato
   mu sync.Mutex
   n  int
}
func (c *Contador) Inc() {
   c.mu.Lock()
   defer c.mu.Unlock()
   c.n++
}
```

- Sin mutex: data race. Detéctala con `go run -race .`
- No copies structs con mutex: receptor siempre `*T`.
- Canales para pasar datos; mutex para proteger estado.

15 strings y strconv

```
strings.ToUpper(s)   strings.ToLower(s)
strings.Contains(s, "x")  strings.HasPrefix(s, "a")
strings.Split("a,b", ",")      // []string{"a","b"}
strings.Join(partes, "-")
strings.TrimSpace(s)  strings.Replace(s, "a", "b", -1)
len(s)                         // bytes, no caracteres
[]rune(s)                      // caracteres Unicode
```

- `Atoi` / `Itoa` en strconv: siempre revisa el `err` de `Atoi`.

16 Trampas comunes

- `:=` solo dentro de funciones; en un bloque interior crea otra variable (shadowing).
- `Printf` no agrega `\n`.
- `s = append(s, x)`: siempre reasigna.
- `make([]int, 3)` ya trae 3 ceros: usa `make([]int, 0, 3)` para `append`.
- Rebanar comparte memoria: copia con `copy` o `slices.Clone`.
- `Unmarshal` necesita `&`.
- `err == ErrX` falla si está envuelto: usa `errors.Is`.
- Escribir en un mapa `nil` hace panic.
- Mapas y contadores compartidos entre goroutines: mutex.
- Esperar goroutines con `Sleep` es un parche: usa `WaitGroup`.
- Desde Go 1.22 cada vuelta de `for` tiene su propia variable (ya no se comparte en closures).

17 Convenciones

- Siempre `gofmt` y `go vet` antes de commitear.
- Mayúscula = exportado. Paquetes en minúscula, sin guiones bajos.
- Errores: variable `err`; mensajes en minúscula, sin punto.
- Interfaces pequeñas; nombres de un método terminan en -er (`Reader`).
- Comentario de doc empieza con el nombre: `// Area calcula...`.
- Nombres cortos en ámbitos cortos (`i`, `s`, `err`); largos en los amplios.
- Acepta interfaces, devuelve structs.